

Cloud Computing & Container Cheat Sheet

1. Cloud Computing

Wie wird Cloud Computing definiert?

Cloud Computing ermöglicht den einfachen, bedarfsorientierten Netzwerkzugriff auf gemeinsam genutzte IT-Ressourcen (Server, Speicher, Netzwerk, Anwendungen), die schnell bereitgestellt und wieder freigegeben werden können.

Welche 5 Merkmale hat eine Cloud?

1. On-Demand Self-Service

- Ressourcen selbst bereitstellen
- Kein Kontakt zum Anbieter nötig

2. Broad Network Access

- Zugriff über Netzwerk / Internet
- Von verschiedenen Geräten möglich

Cloud-Schnittstellen

- CLI
- WebUI
- API

3. Resource Pooling

- Gemeinsame Ressourcennutzung
- Multi-Tenancy
- Standortunabhängigkeit
- Economy of Scale
- Overcommitment

4. Rapid Elasticity

- Schnelles Hoch- und Runterskalieren

5. Measured Service

- Verbrauch wird gemessen
 - Pay-as-you-go
-

Welche Deployment-Modelle gibt es?

Private Cloud

- Nur ein Kunde
- Volle Kontrolle
- Eigene Hardware möglich

Public Cloud

- Für alle Kunden verfügbar
- Kein Zugriff auf Hardware/Rechenzentrum

Community Cloud

- Mehrere Organisationen mit gleichem Interesse

Hybrid Cloud

- Kombination mehrerer Cloud-Typen
-

Welche Service-Modelle gibt es?

IaaS (Infrastructure as a Service)

Du verwaltest:

- Betriebssystem
- Anwendungen
- Daten

Provider verwaltet:

- Netzwerk
- Storage
- Server
- Virtualisierung

PaaS (Platform as a Service)

Du verwaltest:

- Anwendungen
- Daten

Provider verwaltet zusätzlich:

- Runtime
- Middleware
- Betriebssystem

SaaS (Software as a Service)

Provider verwaltet alles.

Beispiele:

- Microsoft 365
- Gmail
- Netflix

FaaS (Function as a Service)

Einzelne Funktionen werden ausgeführt.

Beispiele:

- AWS Lambda
- Azure Functions
- Google Cloud Functions

Was bedeutet Shared Responsibility?

Provider

Security OF the Cloud

- Gebäude
- Hardware
- Netzwerk
- Hypervisor

Kunde

Security IN the Cloud

- Daten
- Benutzer
- Berechtigungen
- Anwendungen

Merksatz:

Je höher die Abstraktion (SaaS), desto weniger Verantwortung trägt der Kunde.

2. Cloud-Dienstleistungen & Anbieter

Welche Cloud-Dienstleistungen gibt es?

- Compute
 - Storage
 - Databases
 - Networking
 - Container Services
 - AI/ML Services
-

Welche Cloud-Anbieter gibt es?

Hyperscaler

- AWS
- Microsoft Azure
- Google Cloud Platform (GCP)

Europäische Anbieter

- Hetzner
 - OVHcloud
 - Swisscom Cloud
-

AWS vs Azure vs GCP

AWS	Azure	GCP
Marktführer	Microsoft Integration	Kubernetes & AI
Grösstes Angebot	Hybrid Cloud	Big Data
S3	Azure OpenAI	Vertex AI

Monitoring vs Logging

Monitoring

- Echtzeitdaten
- CPU
- RAM
- Netzwerk

Logging

- Einzelne Events
- Fehler
- Logins
- Abstürze

Merksatz

Monitoring zeigt **DASS** ein Problem existiert.

Logging zeigt **WARUM** es existiert.

Vorteile der Cloud

- Keine Hardware kaufen
 - Schnelle Bereitstellung
 - Hohe Skalierbarkeit
 - Hohe Verfügbarkeit
 - Pay-as-you-go
 - Weniger Wartung
-

Nachteile der Cloud

- Internetabhängigkeit
 - Vendor Lock-In
 - Datenschutz
 - Kostenkontrolle
 - Shared Responsibility
-

3. Container-Technologie

Was ist Container-Technologie?

Container verpacken Anwendungen inklusive Abhängigkeiten in isolierten Umgebungen.

Container teilen sich den Kernel des Host-Betriebssystems.

Vorteile von Containern gegenüber VMs

- Schneller Start
 - Weniger Ressourcenverbrauch
 - Hohe Effizienz
 - Hohe Portabilität
-

Nachteile von Containern

- Schwächere Isolation
 - Gleicher Kernel notwendig
-

Bekannte Produkte

Container Engines

- Docker
- Podman
- containerd
- CRI-O

Container-Orchestrierung

- Kubernetes
- Docker Swarm
- OpenShift

Virtuelle Maschinen

- VMware
 - Hyper-V
 - KVM
 - VirtualBox
 - Proxmox
-

VM vs Container

Kriterium	VM	Container
Kernel	eigener	geteilt
Startzeit	Minuten	Sekunden
Speicher	GB	MB
Effizienz	geringer	höher
Portabilität	begrenzt	sehr hoch

Können VMs immer durch Container ersetzt werden?

Nein.

VMs werden benötigt bei:

- Unterschiedlichen Betriebssystemen
 - Kernel-Modifikationen
 - Höchster Isolation
 - Direktem Hardwarezugriff
-

Image, Container, Registry

Image

- Vorlage
- Read-Only
- Bauplan

Container

- Laufende Instanz eines Images

Registry

- Speicherort für Images

Beispiele:

- Docker Hub
 - GitHub Container Registry
-

Was ist ein Dockerfile?

Bauplan für den Build-Prozess eines Images.

Änderungen werden im Dockerfile gemacht, nicht im laufenden Container.

Warum gelten Container als immutable?

Container werden nicht verändert.

Änderungen:

1. Neues Image bauen
2. Alten Container löschen
3. Neuen Container starten

Vorteile:

- Reproduzierbarkeit
 - Einfaches Rollback
 - Keine Configuration Drift
-

Self-Managed vs Fully Managed

Self-Managed

Vorteile:

- Volle Kontrolle
- Weniger Vendor Lock-In

Nachteile:

- Hoher Wartungsaufwand
- Expertenwissen nötig

Fully Managed

Vorteile:

- Wenig Betriebsaufwand
- Automatische Updates
- Automatische Skalierung

Nachteile:

- Höhere Kosten
 - Anbieterabhängigkeit
-

4. Container-Orchestrierung

Warum braucht man Container-Orchestrierung?

Verwaltung vieler Container.

Aufgaben:

- Scheduling
 - Auto-Healing
 - Load Balancing
 - Traffic Routing
 - Skalierung
-

Wie funktioniert Container-Orchestrierung?

Soll-Zustand definieren

Beispiel:

Es sollen immer 3 Container laufen.

Ist-Zustand überwachen

Kontinuierlicher Vergleich.

Auto-Healing

Abgestürzte Container werden automatisch ersetzt.

Bekannte Technologien

Kubernetes (K8s)

- Industriestandard
- Marktführer

OpenShift

- Kubernetes-Erweiterung für Unternehmen

Docker Swarm

- Einfacher als Kubernetes

Amazon ECS

- AWS-Alternative
-

Horizontale Skalierung

Scale Out

- Mehr Instanzen hinzufügen

Scale Up

- Mehr CPU / RAM

Cloud-Anwendungen bevorzugen:

Horizontale Skalierung

Deployment-Strategien

Recreate

- Alte Version stoppen
- Neue Version starten
- Downtime

Rolling Update

- Schrittweiser Austausch
- Standard

Blue-Green

- Zwei Umgebungen
- Einfaches Rollback

Canary

- Neue Version zuerst für wenige Benutzer
-

Infrastructure as Code (IaC)

Infrastruktur wird als Code beschrieben.

Tool:

- Terraform
-

5. Git & Docker

Git Workflow

```
git clone <url>
git pull

git add .

git commit -m "message"

git push
```

Docker Workflow

Image bauen

```
docker build -f Dockerfile.web -t ghcr.io/<user>/html-docker-app:1.0.0 .
```

Image hochladen

```
docker push <tag>
```

Container starten

```
docker run -d -p 8080:80 --name web <tag>
```

Container stoppen

```
docker stop <name>
```

Container löschen

```
docker rm <name>
```

Image löschen

```
docker rmi <image>
```

Container anzeigen

```
docker ps -a
```

Logs anzeigen

```
docker logs <name>
```

Shell öffnen

```
docker exec -it <name> sh
```

Alternative:

```
docker exec -it <name> bash
```

6. Prüfungs-Merksätze

- Monitoring = Echtzeitdaten
- Logging = Events
- Container = immutable
- Dockerfile = Bauplan für Image
- Image = Vorlage
- Container = laufende Instanz
- Registry = Speicherort für Images
- Scale Out = horizontal
- Scale Up = vertikal
- Provider = Security OF the Cloud
- Kunde = Security IN the Cloud
- Kubernetes = Scheduling + Load Balancing + Auto-Healing + Scaling
- OpenShift = Kubernetes für Unternehmen